UNIVERSIDADE DO VALE DO ITAJAÍ – UNIVALI
Programa de Pós-graduação em Computação Aplicada – PPGCA
Disciplina de Análise de Algoritmos – semestre 2026/1
Professor Dr. Rudimar Luís Scaranto Dazzi
Acadêmicos: Katia Lorenz, Letícia Zorzi Rama e Ricardo Arruda Júnior

Análise de complexidade de algoritmos de Força bruta e Aproximativo na solução do Problema do Caixeiro Viajante (PCV)

Contextualização

Atividade avaliativa da disciplina de Análise de Algoritmos do Mestrado em Computação Aplicada da Universidade do Vale do Itajaí (UNIVALI), período letivo 2026/01.

O Problema do Caixeiro Viajante (PCV) é um método bem explorado para problemas de otimização computacional. No PCV, objetiva-se encontrar a rota mais curta que visite um conjunto de cidades exatamente uma vez e depois retorne à cidade de origem. Devido à sua classificação como um problema NP-difícil, gerar soluções ótimas para instâncias grandes continua sendo um desafio (GOEL et al., 2025). Para lidar com isso, algoritmos heurísticos têm sido explorados, pois fornecem soluções quase ótimas dentro de tempos razoáveis. Sendo assim, o presente relatório analisa e compara a complexidade de um algoritmo heurístico e um de força bruta quando implementados para se solucionar o Problema do Caixeiro Viajante (PCV) com *datasets* de diferentes tamanhos.

Introdução

O algoritmo de força bruta é considerado o método mais simples para resolver o PCV. O método de força bruta consiste em percorrer todos os percursos possíveis um por um, calcular o comprimento total de cada percurso e encontrar o percurso mínimo desses valores de comprimento usando comparações (NAHER, 2011). Ou seja, fornece o percurso ótimo.

Entretanto, o custo desse método envolve um aumento extremamente rápido com o aumento do número de cidades. O custo do algoritmo de

força bruta fica evidente quando se verifica que existem $(n - 1)! = 1 \cdot 2 \cdot 3 \cdot 4 \cdots (n - 1)$ maneiras diferentes de visitar n cidades em um percurso. Cada percurso começa em uma cidade (por exemplo, a cidade A), depois visita todas as $n-1$ cidades restantes em ordem arbitrária e, finalmente, retorna à cidade A. No entanto, existem exatamente $(n - 1)!$ ordenações ou permutações possíveis das $n - 1$ cidades restantes (NAHER, 2011).

Sendo assim, o algoritmo de força bruta proporciona uma solução ótima para problemas similares ao Problema do Caixeiro Viajante. Porém, a viabilidade de sua implementação se restringe à problemas com poucos n, caso contrário, torna-se inviável. Outros métodos não ótimos, mas factíveis, ganham destaque para a solução do Problema do Caixeiro Viajante, dentre eles os algoritmos heurísticos.

Algoritmos heurísticos não necessariamente encontram a solução global perfeita, mas sim fornecem respostas viáveis para metas imediatas em um tempo de processamento aceitável. Em outras palavras, são definidos como estratégias práticas ou factíveis utilizadas para resolver problemas complexos ou NP-difíceis (GOEL et al., 2025). Exemplos de algoritmos heurísticos envolvem o vizinho mais próximo, a inserção com o menor encargo, a inserção com maior afastamento e métodos aproximativos como árvores geradoras mínimas.

A árvore geradora mínima é um dos problemas de grafos mais antigos e básicos da ciência da computação. Sua história remonta ao algoritmo de Boruvkas. Dado um grafo ponderado por arestas, a árvore geradora mínima consiste em encontrar uma subárvore que abranja todos os vértices e cujo peso total seja mínimo (PEER & WIGDERSON, 1998). Uma árvore geradora mínima é uma árvore formada a partir de um subconjunto das arestas em um dado grafo não direcionado, com duas propriedades: ela gera o grafo, ou seja, inclui todos os vértices, e ela é mínima, ou seja, o peso total de todas as arestas é o menor possível (WU & CHAO, 2003).

Para o Problema do Caixeiro Viajante, a árvore geradora mínima conecta todas as cidades com o menor custo total possível. Para isso, utilizam-se algoritmos clássicos como Prim ou Kruskal, seguida por uma travessia em profundidade para construir uma aproximação de um circuito hamiltoniano (GOEL et al., 2025). A árvore geradora mínima quer achar a melhor de todas as rotas, gerando um grafo euleriano que conecta todas

as cidades e caracteriza-se por ter razão de desempenho no pior caso igual a 2. Ter razão de desempenho no pior caso igual à dois significa que o custo obtido pelo algoritmo da árvore geradora mínima é duas vezes o custo de soluções ótimas como a força bruta (SOTO, 1999).

Algoritmos implementados e Análise de complexidade (Notação Big(O))

Foram implementadas duas abordagens para demonstrar o *trade-off* entre a qualidade da solução e o tempo de execução.

1. Algoritmo de Força Bruta

- **Estratégia e Implementação:** Utiliza um algoritmo recursivo de *backtracking* para verificar exaustivamente todos os caminhos possíveis.
- **Garantia:** Encontra sempre a melhor solução (ótima).
- **Análise de Complexidade: O(n!):**
    - **Comportamento Teórico:** Apresenta um crescimento fatorial (exponencial). O número de permutações escala de maneira insustentável: para 10 cidades, são 3.628.800 permutações possíveis; para 26 cidades, esse número explode para 403.291.461.126.605.635.584.000.000 de operações.
    - **Avaliação:** Devido a essa complexidade, a abordagem torna-se inviável para instâncias com mais de 11 a 13 cidades.

2. Algoritmo Heurístico (Aproximativo)

Estratégia e Implementação: Utiliza uma árvore geradora mínima com a biblioteca SciPy, seguida de uma busca em profundidade (DFS) para construir um ciclo hamiltoniano.

Garantia: Trata-se de um algoritmo 2-aproximado, garantindo uma solução que é, no máximo, 2 vezes pior que o resultado ótimo (ou seja, $\leq$ 2.0x).

Análise de Complexidade: $O(n^2)$ (composto também por passos O(n log n)):

- **Comportamento Teórico:** Apresenta um crescimento polinomial, sendo mais tratável do que a busca exaustiva. Para 10 cidades, realiza apenas cerca de 100 operações; para 26 cidades, 676 operações.

- **Avaliação:** É escalável para resolver instâncias maiores (milhares de cidades) em tempos da ordem de milissegundos.

Testes e Indicadores

Os testes avaliaram conjuntos contendo 5, 9, 10, 11, 12, 13 e um *dataset* completo composto de distâncias rodoviárias entre todas as 26 capitais brasileiras (Macapá ficou de fora por não possuir acesso rodoviário).

Os resultados de desempenho, incluindo o número de operações teóricas executadas, o custo e o *speedup*, estão sumarizados na Tabela 1:

Tabela 1. Resultados de desempenho para diferentes tamanhos do dataset. Legenda: Operações = Op. / Força Bruta = FB / Aproximativo = A / Tempo = T / *Speedup* = Desempenho. Fonte: os autores, 2026.

Número cidades	**Op. (FB)**	**Op. (A)**	**Custo ótimo (FB)**	**T (s) (FB)**	**Custo (A) (Melhor caso)**	**T (s) (A)**	**Desvio em relação ao ótimo**	**Razão**	***Speedup***
5	120	25	575.895	0,000154	575.895	0,018393	-	1,00x	0,01x mais rápido
9	362.880	81	927.306	0,201760	990.337	0,111504	6,80% pior	1,07x	1,81x mais rápido
10	3.628.800	100	1.420.035	1,667059	1.605.210	0,012362	13,04% pior	1,13x	134,85x mais rápido
11	39.916.800	121	1.366.141	15,714983	1.697.592	0,011784	24,26% pior	1,24x	1.333,58x mais rápido
12	479.001.600	144	1.347.316	368,696655	1.607.211	0,019165	19,29% pior	1,2x	19.238,46x mais rápido
13	6.227.020.800	169	1.326.012	2.980,205073	1.468.980	0,015631	10,78% pior	1,11x	190.654,58x mais rápido
26	403.291.461.126.605.635.584.000.0	676	**Inviável**	-	2.361.634	0,136301	-	-	-

Análise dos resultados

A partir dos dados coletados, evidencia-se a relação de compromisso entre tempo e precisão:

1. ***Datasets* Pequenos (n ≤ 12 cidades):** A força bruta é recomendada quando se precisa da solução exata, pois encontra a solução ótima com tempos aceitáveis (variando de frações de milissegundos até cerca de 15 segundos para 11 cidades). Na menor instância (5 cidades), inclusive, a Força Bruta chegou a ser mais veloz que o Aproximativo.
2. ***Datasets* Grandes (Velocidade Crítica):** A partir de 12 e 13 cidades, o tempo da força bruta explode (368 segundos e 2.980 segundos, respectivamente). Para o *dataset* de 26 cidades, o algoritmo mostrou-se inviável. Nesses casos, o algoritmo aproximativo ofereceu uma solução rápida (menos de 0,14 segundos para 26 cidades).

O custo viável do algoritmo de força bruta para *datasets* pequenos e inviável para *datasets* grandes resultado neste presente trabalho vai de encontro aos resultados de Naher (2011). Para o autor, o processamento do algoritmo de força bruta de um único percurso leva cerca de um milissegundo enquanto que um percurso com 16 cidades levaria cerca de 20 anos para resolver um Problema do Caixeiro Viajante. Ademais, tempos de execução que não excedam uma hora só são possíveis para problemas com, no máximo, 10 cidades (NAHER, 2011). Os resultados do autor para o máximo 10 cidades são similares aos resultados do presente estudo os quais consideraram que o algoritmo de força bruta deixa de ser recomendado a partir de 13.

3. **Qualidade da Solução Heurística:** A garantia teórica de que a heurística entregaria um custo, no máximo, 2x maior que o ótimo foi corroborada com Soto (1999). O pior desvio ocorreu no cenário de 11 cidades (24,26% pior que o ótimo, ou razão de 1,2426x), demonstrando uma perda de qualidade considerada pequena diante do enorme ganho de velocidade.

Na revisão de Goel et al. (2025) para diferentes algoritmos heurísticos que solucionem o PCV, a árvore geradora mínima mostrou-se superior aos algoritmos heurísticos vizinho mais próximo e a inserção com maior afastamento em termos de custo-benefício, especialmente para instâncias maiores. Ao aplicar os algoritmos heurísticos vizinho mais próximo, inserção com maior afastamento e a aproximação por árvore geradora mínima a conjuntos de dados de 50, 100, 200 e 1000 cidades, observou-se que o aumento na escala do problema impacta negativamente a eficiência das duas primeiras abordagens. Enquanto o custo do trajeto nos métodos vizinho mais próximo e inserção com maior afastamento apresentou uma elevação significativa — com o primeiro saltando de 688,47 para 2.843,17 e o segundo de 596,28 para 2.568,42 no intervalo de 50 a 1000 cidades. Em contrapartida, o algoritmo de aproximação por árvore geradora mínima manteve custos de trajeto notavelmente baixos e estáveis, apresentando inclusive, uma leve redução de 49,60 para 15,35 na maior instância testada (Tabela 2).

Tabela 2. Comparação de desempenho de algoritmos heurísticos para o problema do caixeiro viajante. Legenda: Nearest Neighbour = Vizinho mais próximo / Farthest Insertion = Inserção com maior afastamento / MST Approximation = Árvore geradora mínima aproximativa. Fonte: adaptado de Goel et al., 2025.

Number of Cities	Nearest Neighbour	Farthest Insertion	MST Approximation
50	688.47	596.28	49.60
100	894.68	772.55	46.59
200	1319.18	1190.66	41.50
1000	2843.17	2568.42	15.35

Esse comportamento evidencia a escalabilidade da árvore geradora mínima e sua capacidade de manter a qualidade da solução mesmo com a expansão considerável do tamanho do problema.

Aplicações Práticas: Soluções eficientes para o PCV, como a a árvore geradora mínima, são fundamentais em áreas como design de circuitos eletrônicos, conexão de ilhas, agrupamento de dados de expressão gênica, design de redes de comunicação e transporte, biologia computacional, alinhamento de múltiplas sequências biológicas e construção de árvores evolutivas e filogenia (WU & CHAO, 2003).

Conclusão

Os experimentos expuseram, na prática, a teoria da complexidade. A notação **O(n!)** e sua explosão de operações demonstram claramente por que o algoritmo de força bruta se torna impraticável conforme os dados aumentam, inviabilizando testes com 26 cidades. Por outro lado, a complexidade polinomial **$O(n^2)$**, com número de operações drasticamente menor, torna o algoritmo aproximativo altamente escalável. Diante dos indicadores obtidos, conclui-se que abordagens heurísticas entregam uma excelente relação custo-benefício para problemas reais.

Referências

GOEL, M.; SINGH, J.; BAIRWA, A. K. Performance Evaluation of Heuristic and Meta-Heuristic Algorithms on Large-Scale Travelling Salesman Problem Instances. **IEEE Acces**s, [S. l.], 5 set. 2025. DOI: 10.1109/ACCESS.2025.3606531. Disponível em: https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=11152317. Acesso em: 26 abr. 2026.

NÄHER, S. **The Travelling Salesman Problem.** In: VÖCKING, B. et al. (Ed.). Algorithms Unplugged. Berlin: Springer, 2011. Disponível em: https://doi.org/10.1007/978-3-642-15328-0_40. Acesso em: 26 abr. 2026.

PEER, D.; WIGDERSON, A. **On Minimum Spanning Trees**. [S. l.: s. n.], 1998. Disponível em: https://www.math.ias.edu/~avi/STUDENTS/dpthesis.pdf. Acesso em: 26 abr. 2026.

SOTO, A. R. P. **Algoritmos Aproximativos para o Problema do Caixeiro Viajante** [Rio de Janeiro] 1999 XII, 87 pp., 29.7 em, (COPPE/UFRJ, M.Sc., Engenharia de Sistemas e Computação, 1999) Tese - Universidade Federal do Rio de Janeiro, COPPE. Disponível em: https://pesc.coppe.ufrj.br/uploadfile/publicacao/775.pdf. Acesso em: 26 abr. 2026.

WU, B. Y.; CHAO, K. **Spanning trees and optimization problems**. Boca Raton, FL: Chapman & Hall/CRC Press, 2003. Disponível em: https://www.csie.ntu.edu.tw/~kmchao/books/treebook/contents.pdf. Acesso em: 26 abr. 2026.